

Министерство науки и высшего образования Российской Федерации
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №2

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Оценка времени выполнения программ»

Выполнили:

студенты групп 24ВВВ3

Савин А. В.

Назаров Н. Д.

Майер В. С.

Приняли:

Юрова О. В.

Деев М. В.

Пенза 2025

Цель работы

Освоить на практике методы оценки времени выполнения программ на языке C с использованием библиотеки `time.h`. Закрепить навыки измерения временной сложности алгоритмов и анализа их производительности.

Лабораторное задание

Задание 1:

1. Вычислить порядок сложности программы (O -символику).
2. Оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки `time.h` для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000.
3. Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

Задание 2:

1. Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива.
2. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел.
3. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел.
4. Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую.
5. Оценить время работы стандартной функции `qsort`, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

Работа программы

Консоль отладки Microsoft Visual Studio

Задание 1: Умножение матриц

Размер	Время (сек)	$O(n^3)$
100	0,004000	$O(1000000)$
200	0,019000	$O(8000000)$
500	0,491000	$O(125000000)$
1000	5,923000	$O(1000000000)$
5000	708,402000	$O(125000000000)$
10000	57823,266000	$O(1000000000000)$

Выполнение задания 1



График зависимости времени выполнения

Тип: Случайные				
Размер	Шелл (мс)	QS (мс)	qsort (мс)	
5000	2,00	0,00	1,00	
6000	1,00	1,00	1,00	
7000	2,00	1,00	1,00	
Тип: Возрастающая				
Размер	Шелл (мс)	QS (мс)	qsort (мс)	
5000	1,00	0,00	0,00	
6000	0,00	1,00	0,00	
7000	0,00	0,00	1,00	
Тип: Убывающая				
Размер	Шелл (мс)	QS (мс)	qsort (мс)	
5000	2,00	0,00	1,00	
6000	3,00	0,00	1,00	
7000	4,00	0,00	1,00	
Тип: Половина				
Размер	Шелл (мс)	QS (мс)	qsort (мс)	
5000	1,00	5,00	2,00	
6000	2,00	7,00	2,00	
7000	2,00	10,00	3,00	

Вывод

Освоили на практике методы оценки времени выполнения программ на языке C с использованием библиотеки `time.h`. Закрепили навыки измерения временной сложности алгоритмов умножения матриц и различных методов сортировки. Проанализировали зависимость времени выполнения от размера входных данных и типа алгоритма. Написали рабочий код, реализующий сравнение алгоритмов и измерение их производительности.

Листинг

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#include <locale.h>

void shell(int* items, int count)
{

    int i, j, gap, k;
    int x, a[5];

    a[0] = 9; a[1] = 5; a[2] = 3; a[3] = 2; a[4] = 1;

    for (k = 0; k < 5; k++) {
        gap = a[k];
        for (i = gap; i < count; ++i) {
            x = items[i];
            for (j = i - gap; (x < items[j]) && (j >= 0); j = j - gap)
                items[j + gap] = items[j];
            items[j + gap] = x;
        }
    }
}

void qs(int* items, int left, int right) {
    int i, j;
    int x, y;
```

```

i = left; j = right;

x = items[(left + right) / 2];

do {
    while ((items[i] < x) && (i < right)) i++;
    while ((x < items[j]) && (j > left)) j--;

    if (i <= j) {
        y = items[i];
        items[i] = items[j];
        items[j] = y;
        i++; j--;
    }
} while (i <= j);

if (left < j) qs(items, left, j);
if (i < right) qs(items, i, right);
}

int compare(const void* a, const void* b) {
    return (*(int*)a - *(int*)b);
}

int* create_array(int size, int type) {
    int* arr = (int*)malloc(size * sizeof(int));
    if (arr == NULL) return NULL;

    switch (type) {
        case 0:
            for (int i = 0; i < size; i++) {
                arr[i] = rand() % 1000;
            }
            break;
        case 1:
            for (int i = 0; i < size; i++) {

```

```

        arr[i] = i;
    }
    break;
case 2:
    for (int i = 0; i < size; i++) {
        arr[i] = size - i;
    }
    break;
case 3:
    for (int i = 0; i < size / 2; i++) {
        arr[i] = i;
    }
    for (int i = size / 2; i < size; i++) {
        arr[i] = size - i;
    }
    break;
}
return arr;
}

```

```

int* copy_array(int* src, int size) {
    int* dst = (int*)malloc(size * sizeof(int));
    if (dst == NULL) return NULL;
    for (int i = 0; i < size; i++) {
        dst[i] = src[i];
    }
    return dst;
}

```

```

int** create_matrix(int n) {
    int** matrix = (int**)malloc(n * sizeof(int*));
    if (matrix == NULL) return NULL;

    for (int i = 0; i < n; i++) {
        matrix[i] = (int*)malloc(n * sizeof(int));
        if (matrix[i] == NULL) {

```

```

        for (int j = 0; j < i; j++) {
            free(matrix[j]);
        }
        free(matrix);
        return NULL;
    }
}

return matrix;
}

void free_matrix(int** matrix, int n) {
    for (int i = 0; i < n; i++) {
        free(matrix[i]);
    }
    free(matrix);
}

void task1() {
    int sizes[] = { 100, 200, 500, 1000, 5000, 10000 };
    int num_sizes = sizeof(sizes) / sizeof(sizes[0]);

    printf("Задание 1: Умножение матриц\n");
    printf("Размер | Время (сек) | O(n^3)\n");
    printf("-----\n");

    for (int s = 0; s < num_sizes; s++) {
        int n = sizes[s];

        if (n > 10000) {
            printf("%6d | Превышен лимит размера\n", n);
            continue;
        }

        int** a = create_matrix(n);
        int** b = create_matrix(n);
        int** c = create_matrix(n);
    }
}

```

```

if (a == NULL || b == NULL || c == NULL) {
    printf("%6d | Ошибка выделения памяти\n", n);
    if (a) free_matrix(a, n);
    if (b) free_matrix(b, n);
    if (c) free_matrix(c, n);
    continue;
}

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        a[i][j] = rand() % 100 + 1;
        b[i][j] = rand() % 100 + 1;
    }
}

clock_t start = clock();

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        int elem_c = 0;
        for (int r = 0; r < n; r++) {
            elem_c += a[i][r] * b[r][j];
        }
        c[i][j] = elem_c;
    }
}

clock_t end = clock();
double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;

printf("%7d | %14.6f | O(%lld)\n", n, time_taken, (long long)n * n * n);

free_matrix(a, n);
free_matrix(b, n);
free_matrix(c, n);

```



```

    }
    printf("\n");
}

void task2() {
    int sizes[] = { 5000, 6000, 7000};
    const char* types[] = { "Случайные", "Возрастающая", "Убывающая", "Половина" };

    printf("Задание 2: Сравнение алгоритмов сортировки\n");

    for (int type = 0; type < 4; type++) {
        printf("\nТип: %s\n", types[type]);
        printf("Размер    |  Шелл (мс) |    QS (мс) | qsort (мс) \n");
        printf("-----\n");

        for (int s = 0; s < 3; s++) {
            int size = sizes[s];
            int* original = create_array(size, type);
            if (original == NULL) {
                printf("%7d | Ошибка создания массива\n", size);
                continue;
            }

            double t_shell = 0, t_qs = 0, t_qsort = 0;

            int* arr1 = copy_array(original, size);
            if (arr1) {
                clock_t start = clock();
                shell(arr1, size);
                clock_t end = clock();
                t_shell = ((double)(end - start)) / CLOCKS_PER_SEC * 1000;
                free(arr1);
            }

            int* arr2 = copy_array(original, size);
            if (arr2) {

```

```

        clock_t start = clock();
        qs(arr2, 0, size - 1);
        clock_t end = clock();
        t_qs = ((double)(end - start)) / CLOCKS_PER_SEC * 1000;
        free(arr2);
    }

    int* arr3 = copy_array(original, size);
    if (arr3) {
        clock_t start = clock();
        qsort(arr3, size, sizeof(int), compare);
        clock_t end = clock();
        t_qsort = ((double)(end - start)) / CLOCKS_PER_SEC * 1000;
        free(arr3);
    }

    printf("%7d | %10.2f | %10.2f | %10.2f\n", size, t_shell, t_qs, t_qsort);
    free(original);
}

}

int main(void) {
    setlocale(LC_ALL, "Russian");

    srand((unsigned int)time(NULL));

    task1();
    task2();

    return 0;
}

```